

挑战杯指导文档

2025挑战杯指导文档

前言

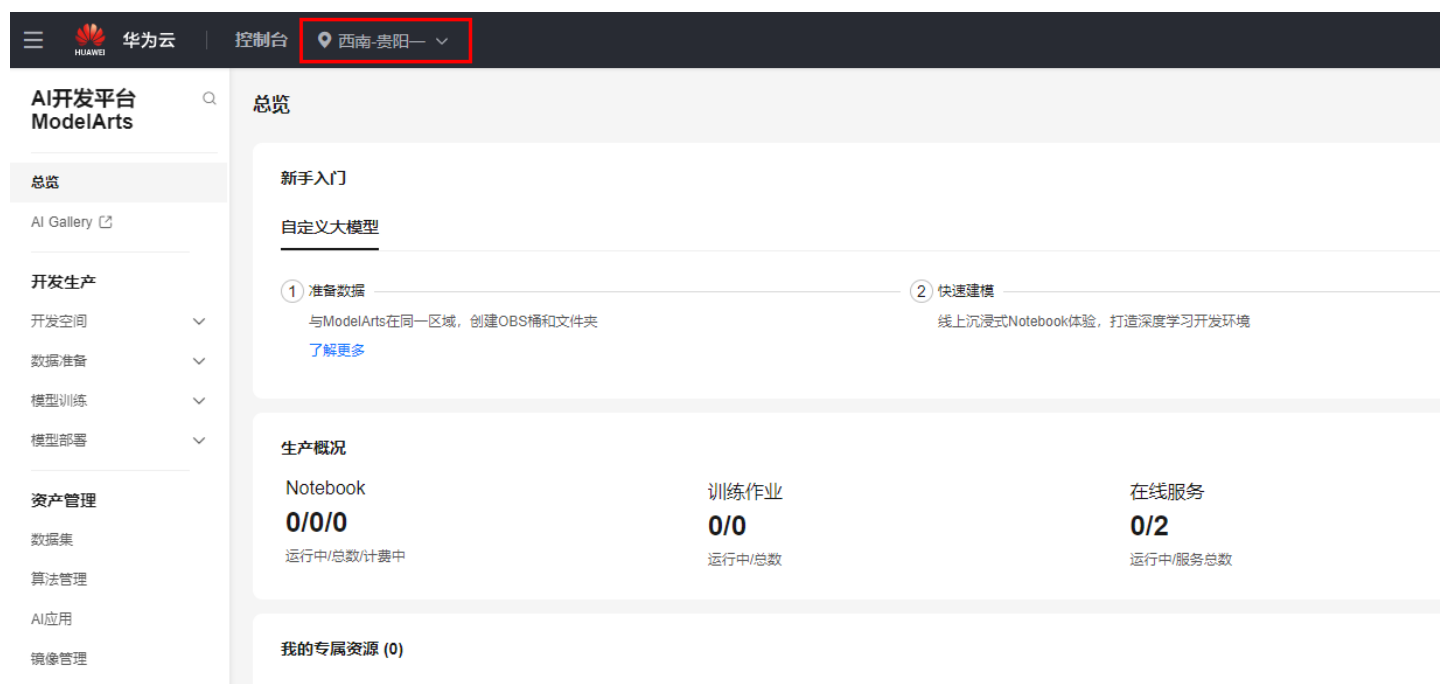
本文档旨在让选手熟悉挑战杯A榜的开发环境，掌握如何测试和提交代码，还可以了解ModelArts平台的基本使用方法。

注意：如何微调模型、调整推理参数、优化推理工程等内容，不属于本教程的范畴。

代码块

```
1  flowchart LR
2      subgraph 1.环境准备
3          direction TB
4          t1[MA访问授权] --> b1[Notebook准备] --> b2[CANN包安装] --> b3[镜像保存]
5      end
6      subgraph 2.代码调试
7          direction TB
8          t2[下载模型] --> t3[准备推理框架] --> t4[测试代码]
9      end
10     subgraph 3.代码优化
11         direction TB
12         c1[微调模型/优化流程\n（选手自行完成）] --> c2[获取测试用例\n推理结果] -->
c3[提交结果文件\n到大赛平台] --> c1[微调模型/优化流程\n（选手自行完成）]
13     end
14     subgraph 4.资源清理
15         direction TB
16         d1[MA资源释放]
17     end
18     1.环境准备 --> 2.代码调试
19     2.代码调试 --> 3.代码优化
20     3.代码优化 --> 4.资源清理
```

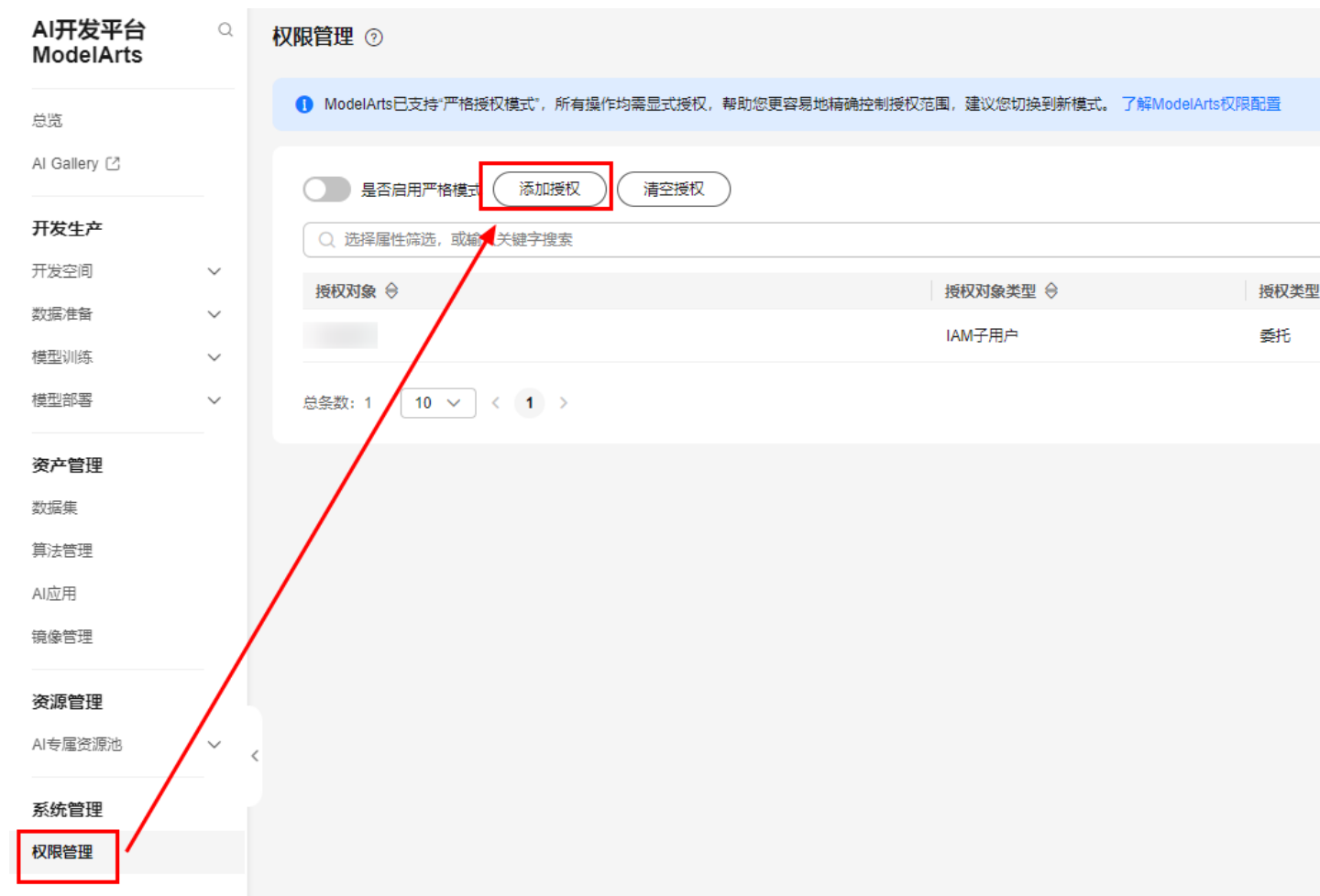
本案例涉及到的所有OBS、ModelArts等资源都应在**贵阳一**区域下运行：



准备环境

我们使用ModelArts对模型及工程进行调试训练，使用OBS存储服务存储数据、模型、代码。在ModelArts上使用Notebook、OBS存储服务、训练作业等等很多功能，都需要先进行访问授权。[点此进入ModelArts控制台](#)，按照以下步骤操作即可。

- 选择左侧导航栏中的权限管理，在右侧界面中点击添加授权：



- 在弹出的添加授权界面中，按照下图所示依次操作：



准备notebook

点击进入ModelArts的开发环境，右上角创建Notebook：



选择对应的镜像、计算规格与存储配置，其中存储配置方面，文件可以放在 `/cache` 目录下，这样不会占用EVS空间，但 `/cache` 目录会在每次启动时刷新。如果想要持久化存储，请对EVS扩容，同时将安装文件存储至 `/home/ma-user/work`，存储会持续产生少量费用（无论是否运行），代金券使用场景覆盖存储。

★ 镜像

公共镜像

自定义镜像

?

前往AI Gallery获取更多镜像

请输入镜像名称

Q

Q

	名称	描述
☐	mindspore_2.2.0-cann_7.0.1-py_3.9-euler_2.10.7-aarch64-snt9b	Ascend Snt9b+ARM algorithm development and training. MindSpore is...
☐	pytorch_1.11.0-cann_6.3.2-py_3.7-euler_2.10.7-aarch64-snt9b	Ascend Snt9b+ARM algorithm development and training. PyTorch is pr...
☐	pytorch1.8-cuda10.2-cudnn7-ubuntu18.04	CPU and GPU general algorithm development and training, preconfigu...
☐	tensorflow2.1-cuda10.1-cudnn7-ubuntu18.04	CPU、GPU通用算法开发和训练基础镜像，预置AI引擎TensorFlow2.1
☐	mindspore_2.1.0-cann_6.3.2-py_3.7-euler_2.10.7-aarch64-snt9b	Ascend Snt9b+ARM algorithm development and training. MindSpore is...
☐	pytorch_1.11.0-cann_8.0.rc1-py_3.9-euler_2.10.7-aarch64-snt9b	Ascend_snt9c+ARM算法开发和训练基础镜像，AI引擎预置PyTorch1.11
☐	pytorch_1.11.0-cann_8.0.rc2-py_3.9-euler_2.10.7-aarch64-snt9b	Ascend_snt9c+ARM算法开发和训练基础镜像，AI引擎预置PyTorch1.11
☐	mindspore_2.3.0-cann_8.0.rc2-py_3.9-euler_2.10.7-aarch64-s...	Ascend Snt9B+ARM算法开发和训练基础镜像，AI引擎预置MindSpore
☐	mindspore_2.3.0-cann_8.0.rc1-py_3.9-euler_2.10.7-aarch64-s...	Ascend Snt9B+ARM算法开发和训练基础镜像，AI引擎预置MindSpore
☒	pytorch_2.1.0-cann_8.0.rc2-py_3.9-euler_2.10.7-aarch64-snt9b	Ascend_snt9b+ARM算法开发和训练基础镜像，AI引擎预置PyTorch2.1

总条数: 13

10 < 1 2 >

★ 资源类型

公共资源池

专属资源池

★ 类型

ASCEND

★ 实例规格

Ascend: 1*ascend-snt9b1|ARM: 24核 192GB

昇腾规格，适合深度学习场景下的模型训练和调测

★ 存储配置

云硬盘EVS

云硬盘EVS作为持久化存储挂载在/home/ma-user/work目录下，该目录下的内容在实例停止后会被保留，存储支持在线按需扩容

★ 磁盘规格

-

50

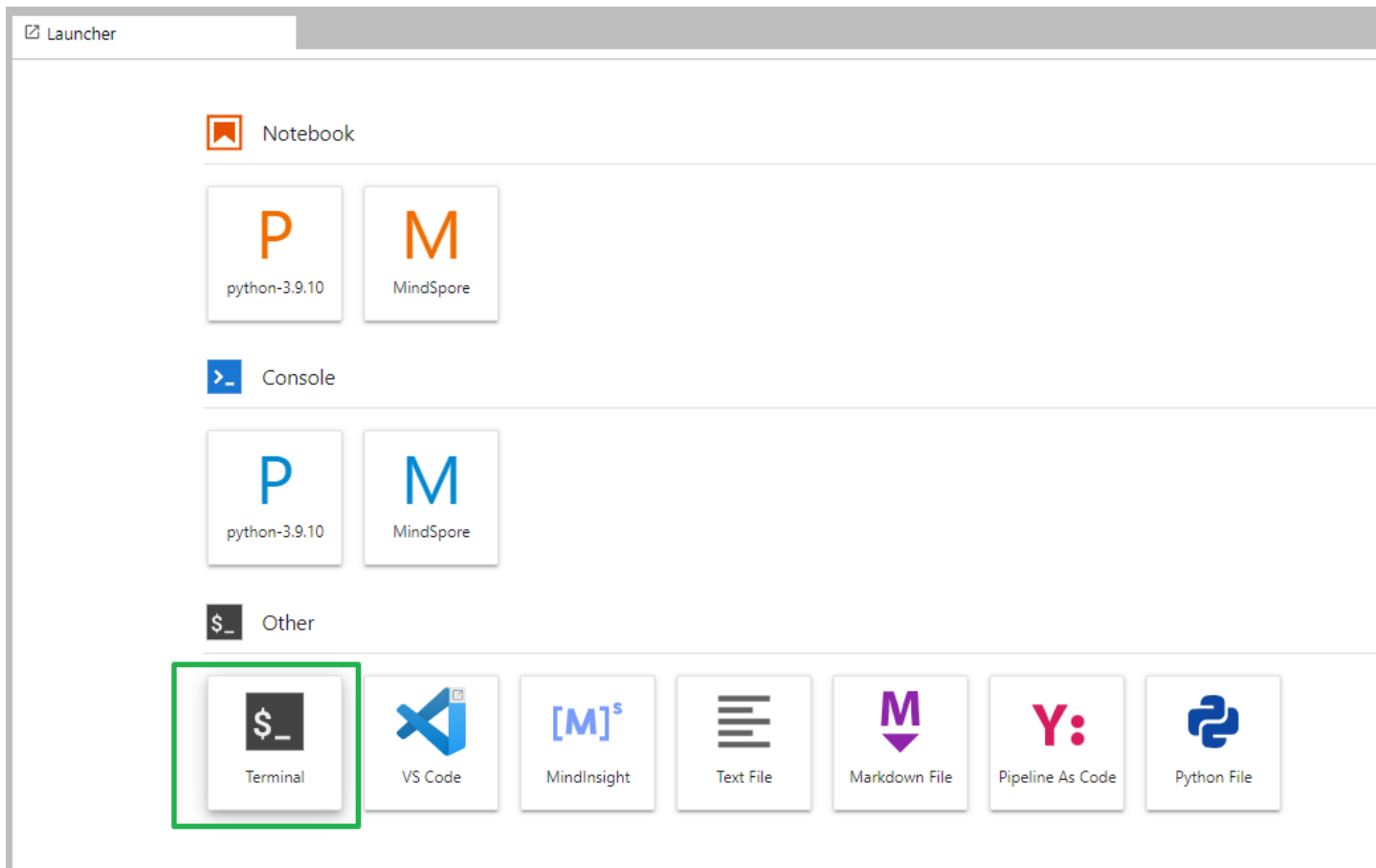
+

GB

磁盘规格默认为5GB，从Notebook实例创建成功起，直至删除成功，每GB按照规定费用收费。

我们可以选择公开镜像：`pytorch_2.1.0-cann_8.0.rc2-py_3.9-euler_2.10.7-aarch64-snt9b`，后续安装了8.1.RC1版本软件包后，可以参考文档 [保存Notebook实例](#) 保存为自定义镜像供后续使用。

创建完成后，点击**打开**进入notebook。为了方便配置环境变量等，我们在Terminal中进行操作，点击**Terminal**进入终端：



切换环境：

代码块

```
1  conda deactivate
2  conda activate python-3.9.10
```

接下来即可安装软件包。

下载并安装CANN软件包

我们的安装包要从 `.myhuaweicloud.com` 域名下载，因此从no_proxy环境变量中删除 `.myhuaweicloud.com`：

代码块

```
1  env | grep no_proxy
```

假设输出为：`no_proxy=127.0.0.1,localhost,172.16.*,obs.cn-southwest-2.myhuaweicloud.com,iam.cn-southwest-2.huaweicloud.com,pip.modelarts.private.com,.myhuaweicloud.com`，我们删除 `.myhuaweicloud.com`：

```
1 export no_proxy=127.0.0.1,localhost,172.16.*,obs.cn-southwest-  
2.myhuaweicloud.com,iam.cn-southwest-  
2.huaweicloud.com,pip.modelarts.private.com
```

下载并安装CANN软件包，默认安装目录为 `$HOME`，软件包较大，可能需要几分钟：

代码块

```
1 wget https://ascend-repo.obs.cn-east-  
2.myhuaweicloud.com/CANN/CANN%208.1.RC1/Ascend-cann-toolkit_8.1.RC1_linux-  
"$ (uname -i)".run  
2 chmod +x ./Ascend-cann-toolkit_8.1.RC1_linux-"$ (uname -i)".run  
3 ./Ascend-cann-toolkit_8.1.RC1_linux-"$ (uname -i)".run --full --quiet
```

出现如下日志，说明安装成功：

代码块

```
1 Toolkit: Ascend-cann-toolkit_8.1.RC1_linux-aarch64 install success, installed  
in /home/ma-user/Ascend.
```

安装成功后，需要激活环境变量：

代码块

```
1 source /home/ma-user/Ascend/ascend-toolkit/set_env.sh
```

下载并安装CANN算子二进制包

代码块

```
1 wget https://ascend-repo.obs.cn-east-  
2.myhuaweicloud.com/CANN/CANN%208.1.RC1/Ascend-cann-kernels-910b_8.1.RC1_linux-  
"$ (uname -i)".run  
2 chmod +x ./Ascend-cann-kernels-910b_8.1.RC1_linux-"$ (uname -i)".run  
3 ./Ascend-cann-kernels-910b_8.1.RC1_linux-"$ (uname -i)".run --install --quiet
```

出现如下日志，说明安装成功：

代码块

```
1 [kernels_910b] [2025xxxx-xx:xx:xx] [INFO] Ascend-cann-kernels-  
910b_8.1.RC1_linux-aarch64 install success. The installation path is /home/ma-
```

```
user/Ascend/ascend-toolkit.
```

下载并安装nnal

代码块

```
1 wget https://ascend-repo.obs.cn-east-2.myhuaweicloud.com/CANN/CANN%208.1.RC1/Ascend-cann-nnal_8.1.RC1_linux-  
"$$(uname -i)".run  
2 chmod +x ./Ascend-cann-nnal_8.1.RC1_linux-"$$(uname -i)".run  
3 ./Ascend-cann-nnal_8.1.RC1_linux-"$$(uname -i)".run --install --quiet
```

出现如下日志，说明安装成功：

代码块

```
1 [NNAL] [2025xxxx-xx:xx:xx] [INFO] Ascend-cann-nnal_8.1.RC1_linux-aarch64.run  
install success
```

安装成功后，需要激活环境变量：

代码块

```
1 source /home/ma-user/Ascend/nnal/atb/set_env.sh
```

准备torch相关环境

代码块

```
1 pip install torch==2.5.1 torch-npu==2.5.1
```

至此，我们已经完成了全部环境准备的工作。

建议将上述source set_env.sh添加到/home/ma-user/.bashrc中。保存为自定义镜像后，后续启动notebook无需再准备环境。

请注意，B榜判分环境使用的软件依赖与上述环境一致，此外选手可以在提交的模型包中配置可以通过pip安装的依赖包。

下载模型

请注意，本次大赛不约束模型结构，但参赛选手只能使用3B及以下的模型。

我们以Qwen2.5-3B为例，使用modelscope下载开源模型：

代码块

```
1 pip install modelscope==1.25.0
2 modelscope download --model "Qwen/Qwen2.5-3B" --local_dir "/home/ma-
  user/work/Qwen2.5-3B"
```

模型保存在 `/home/ma-user/work/Qwen2.5-3B` 目录下。

推理框架

我们使用开源推理框架vllm以及昇腾后端插件vllm-ascend进行推理，昇腾后端插件具体使用可以参考文档：[vLLM Ascend Plugin](#)。

推理框架依赖下载安装至指定文件夹下：

代码块

```
1 mkdir -p requirements
```

安装vllm：

代码块

```
1 cd requirements
2 git clone --depth 1 --branch v0.8.4 https://github.com/vllm-project/vllm
3 cd vllm
4 VLLM_TARGET_DEVICE=empty pip install -v -e .
5 cd ..
```

如果安装vllm最后有报错，请尝试使用：`VLLM_TARGET_DEVICE=empty pip install -v -e . --user`

安装vllm-ascend：

代码块

```
1 git clone --depth 1 --branch v0.8.4rc2 https://github.com/vllm-project/vllm-
  ascend.git
2 cd vllm-ascend
3 COMPILE_CUSTOM_KERNELS=0 pip install -v -e .
4 cd ..
```

查看安装结果：

```
pip list | grep vllm
```

推理测试

准备好推理框架后，我们可以进行测试。

进入 `$WORK_DIR` 目录下，使用离线推理测试：

代码块

```
1 cd $WORK_DIR
2 vim offline_inference.py
```

推理脚本如下：

代码块

```
1 from vllm import LLM, SamplingParams
2
3 prompts = [
4     "Hello, my name is",
5     "The president of the United States is",
6     "The capital of France is",
7     "The future of AI is",
8 ]
9
10 # Create a sampling params object.
11 sampling_params = SamplingParams(temperature=0.8, top_p=0.95)
12 # Create an LLM.
13 llm = LLM(model="/home/ma-user/work/Qwen2.5-3B")
14
15 # Generate texts from the prompts.
16 outputs = llm.generate(prompts, sampling_params)
17 for output in outputs:
18     prompt = output.prompt
19     generated_text = output.outputs[0].text
20     print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

运行：

代码块

```
1 python offline_inference.py
```

结果如下：

```
INFO 05-13 10:21:12 [model_runner.py:950] Starting to load model /home/ma-user/work/Qwen2.5-3B...
Loading safetensors checkpoint shards:   0% Completed | 0/2 [00:00<?, ?it/s]
Loading safetensors checkpoint shards:   50% Completed | 1/2 [00:05<00:05, 5.98s/it]
Loading safetensors checkpoint shards:  100% Completed | 2/2 [00:16<00:00, 8.63s/it]
Loading safetensors checkpoint shards:  100% Completed | 2/2 [00:16<00:00, 8.23s/it]

INFO 05-13 10:21:29 [loader.py:458] Loading weights took 16.83 seconds
INFO 05-13 10:21:30 [model_runner.py:955] Loading model weights took 5.7917 GB
INFO 05-13 10:21:37 [executor_base.py:112] # npu blocks: 3799, # CPU blocks: 910
INFO 05-13 10:21:37 [executor_base.py:117] Maximum concurrency for 32768 tokens per request: 14.84x
INFO 05-13 10:21:38 [llm_engine.py:449] init engine (profile, create kv cache, warmup model) took 8.21 seconds
Processed prompts: 100%|██████████████████████████████████████████████████████████████████████████████| 4/4 [00:00<00:00, 5.75it/s, est. speed input: 31.64 toks/s, output: 92.05 toks/s]
Prompt: 'Hello, my name is', Generated text: ' Niki and I am a Graphic Designer and Artist based in South-East London.'
Prompt: 'The president of the United States is', Generated text: ' the nation's chief executive and head of state. The president's role in the'
Prompt: 'The capital of France is', Generated text: ' Paris. The capital of Spain is Madrid. The capital of Germany is Berlin.'
Prompt: 'The future of AI is', Generated text: ' now. It is a reality. In our world of fast-moving technology, AI'
```

至此，我们已经完成了全部推理的依赖工作。

使用测试用例推理

我们的测试用例是以jsonl格式组织的，每行一条用例，每条用例具体字段如下：

字段	类型	释义
id	字符串	测试用例唯一标识符
type	字符串	测试用例类型，共有 choice、code-generate、generic-generate、math四种类型
prompt	字符串	测试用例内容
choices	字典	choice类型特有，测试用例选项

选手应该以指定格式返回推理结果：

代码块

```
1  {"result": {
2      "results": [
3          {"id": " id1", "content": "xxx"},
4          {"id": " id2", "content": "xxx"}],
5      }
6  }
```

我们会试图在返回内容中按照某种特定格式提取回答，首先，提取"<answer>"标签内内容:

代码块

```
1 format_pattern = re.compile(r'^<think>(.*?)</think>\s*<answer>(.*?)</answer>$', re.DOTALL)
2 format_pattern.findall(content)
```

接下来，根据不同类型问题，会使用不同格式匹配提取，具体类型说明如下：

类型	匹配内容
choice	<code>r"(?i)Answer[\t]*:[\t]*\\$(?([A-D]*)\)?\$"</code>
code-generate	<code>r"`python\n(.*?)`"</code>
generic-generate	<code>r"(?i)Answer\s*:\s*([^\n]+)"</code>
math	<code>r"(?i)Answer\s*:\s*\s*([^\n]+)"</code>

其中，math校验精度时期望大家提供的结果符合latex标准，并包裹在 `\boxed{}` 框内，如 `\\boxed{\\frac{1}{2}}` (`\\` 以防转义)。一个示例如下：

代码块

```
1  {
2      "id": "681c2bbeb449f9165cc85ba5",
3      "content": "Answer: \\boxed{\\frac{\\sqrt{2}}{2} + \\sqrt{6}}{4}}",
4  },
```

对于上述所有格式，如果没有匹配到内容，则会使用原始字符串做精度校验。

我们将结合业界常用的评估方法，给出参考的prompt模板：

类型	参考模板
choice	"Answer the following multiple choice question. The last line of your response should be of the following format: 'Answer: \$LETTER' (without quotes) where LETTER is one of ABCD. Think step by step before answering.\n\n{Question}\n\nA) {A}\nB) {B}\nC) {C}\nD) {D}"
code-generate	"Read the following function signature and docstring, and fully implement the function described. Your response should only contain the code for this function.\n"
generic-generate	"You will be asked to read a passage and answer a question. Think step by step, then write a line of

	the form 'Answer: \$ANSWER' at the end of your response."
math	"Solve the following math problem step by step. The last line of your response should be of the form Answer: \$ANSWER (without quotes) where \$ANSWER is the answer to the problem.\n\n{Question}\n\nRemember to put your answer on its own line after 'Answer:', and indicate your final answer in boxed LaTeX. For example, if the final answer is \sqrt{3}, write it as \boxed{\sqrt{3}}."

特别说明：对于code-generate类型问题，选手可以返回3条结果，我们使用pass@3作为评价指标，如：

代码块

```
1  {
2      "id": "681c2cf3b449f9165cc85baa",
3      "content": ["<think>...</think><answer>    import time\n
time.sleep(5)\n    return 1</answer>", "    return 1", "\treturn 1"]
4  },
```

对于不足3条的结果，我们会使用已有结果随机扩展到3条，对于超过3条的结果，会随机采样3条。
一个测试用例读取与推理结果返回的端到端案例如下：

代码块

```
1  import json
2  from vllm import LLM, SamplingParams
3
4  class Infer:
5      def __init__(self, model="/home/ma-user/work/Qwen2.5-3B"):
6          self.llm = LLM(model=model)
7          self.sampling_params = {
8              "choice": SamplingParams(max_tokens=1024, temperature=0.8,
top_p=0.95),
9              "code-generate": SamplingParams(n=3, max_tokens=2048,
temperature=0.8, top_p=0.95),
10             "generic-generate": SamplingParams(max_tokens=128,
temperature=0.8, top_p=0.95),
11             "math": SamplingParams(max_tokens=512, temperature=0.8, top_p=0.95)
12         }
13         self.prompt_templates = {
```

```

14         "choice": "Answer the following multiple choice question. The last
line of your response should be of the following format: 'Answer: $LETTER'
(without quotes) where LETTER is one of ABCD. Think step by step before
answering.\n\n{Question}\n\nA) {A}\nB) {B}\nC) {C}\nD) {D}",
15         "code-generate": "Read the following function signature and
docstring, and fully implement the function described. Your response should
only contain the code for this function.\n",
16         "generic-generate": "You will be asked to read a passage and
answer a question. Think step by step, then write a line of the form 'Answer:
$ANSWER' at the end of your response.",
17         "math": "Solve the following math problem step by step. The last
line of your response should be of the form Answer: \ $ANSWER (without quotes)
where $ANSWER is the answer to the problem.\n\n{Question}\n\nRemember to put
your answer on its own line after 'Answer:', and indicate your final answer in
boxed LaTeX. For example, if the final answer is  $\sqrt{3}$ , write it as
\boxed{\sqrt{3}}."
18     }
19
20     def infer(self, data_file="A.jsonl"):
21         a_datas = []
22         with open(data_file, 'r', encoding="utf-8") as f:
23             for line in f:
24                 a_data = json.loads(line)
25                 a_datas.append(a_data)
26
27         res = {"result": {"results": []}}
28
29         for a_data in a_datas:
30             type_ = a_data.get("type")
31             id_ = a_data.get("id")
32
33             template = self.prompt_templates[type_]
34             prompt = ""
35
36             if type_ == "choice":
37                 choices = a_data["choices"]
38                 prompt = template.format(Question=a_data["prompt"],
A=choices["A"], B=choices["B"], C=choices["C"], D=choices["D"])
39
40             elif type_ == "math":
41                 prompt = template.format(Question=a_data["prompt"])
42             else:
43                 prompt = template + a_data["prompt"]
44
45             generated_text = []
46             outputs = self.llm.generate(prompt, self.sampling_params[type_])
47             for output in outputs:

```

```
48         for o in output.outputs:
49             generated_text.append(o.text)
50         generated_text = generated_text[0] if len(generated_text) == 1
51     else generated_text
52         res["result"]["results"].append({"id": id_, "content":
53     generated_text})
54
55     return res
56
57 if __name__ == "__main__":
58
59     infer = Infer(model="/home/ma-user/work/Qwen2.5-3B")
60     res = infer.infer(data_file="A.jsonl")
61     with open("res.json", "w", encoding="utf-8") as f:
62         json.dump(res, f, ensure_ascii=False)
```

请注意，该案例仅供参考，选手可以根据自己的优化点组织代码。